

SLIIT
SE3082 – Parallel Computing



REPORT
LAB 05

Akash D
IT24101853

Exercise 03

Code Implementation

```
M /a/3 Semester/PC/Lab05/Exercise03
GNU nano 9.1
#include <omp.h>
#include <stdio.h>

int main() {

#pragma omp parallel
{
    int th = omp_get_thread_num();
    printf("thread # %d\n", th);

#pragma omp master
{
    printf("Inside Master # %d\n", omp_get_thread_num());
    printf("Exiting Master \n");
}

    printf("Hi again from thread # %d\n", omp_get_thread_num());
}

    return 0;
}
```

Output

```
M /a/3 Semester/PC/Lab05/Exercise03

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise02
$ cd "/a/3 Semester/PC/Lab05"

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05
$ cd Exercise03

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise03
$ nano lab05c.c

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise03
$ gcc -fopenmp lab05c.c -o lab05c.o
lab05c.c: In function 'main':
lab05c.c:11:10: warning: 'master' construct deprecated since OpenMP 5.1, use 'masked' [-Wdeprecated-openmp]
   11 | #pragma omp master
      |             ^~~~

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise03
$ ./lab05c.o
thread # 1
Hi again from thread # 1
thread # 4
Hi again from thread # 4
thread # 0
Inside Master # 0
thread # 3
Hi again from thread # 3
thread # 7
Hi again from thread # 7
thread # 11
Hi again from thread # 11
thread # 12
Hi again from thread # 12
thread # 2
Hi again from thread # 2
thread # 6
Hi again from thread # 6
thread # 9
Hi again from thread # 9
thread # 5
Hi again from thread # 5
thread # 8
Hi again from thread # 8
thread # 15
Hi again from thread # 15
Exiting Master
Hi again from thread # 0
thread # 13
Hi again from thread # 13
thread # 10
Hi again from thread # 10
thread # 14
Hi again from thread # 14

LENOVO@Akash MINGW64 /a/3 Semester/PC/Lab05/Exercise03
$
```

In OpenMP `#pragma omp master` specifies that the enclosed code section is executed exclusively by the master thread (thread 0) of the team. Unlike the single construct there is no implicit barrier at the end of a master construct meaning other threads do not wait or synchronize when they reach the end of it and simply continue execution.

In this specific program:

1. **Parallel Execution:** When the parallel region starts multiple threads execute concurrently printing their respective thread numbers.
2. **Master Construct Execution:** Only thread 0 executes the print statements inside the master block showing Inside Master 0 and Exiting Master while all other threads completely bypass this block.
3. **No Implicit Barrier:** Because there is no barrier at the end of the master construct other threads proceed directly to the next instructions independently resulting in non-synchronized interleaving of the subsequent print statements.